

Image Processing & Recognition: Lecture 10 Review

Welcome to Level 10! We have reached the final boss: **Creation**. Instead of just looking at an image and saying "that's a cat" (discriminative), we are now asking the computer to *draw* a cat from scratch (generative).

Part 1: What are you learning? (The "Human" Summary)

1. The Paradigm Shift

- **Discriminative Learning (Past 9 lectures):** $P(y|x)$. Given an image x , predict the label y .
 - Goal: Draw a line separating cats from dogs.
- **Generative Learning (This lecture):** $P(x)$. Learn the distribution of the data itself.
 - Goal: Understand what "cat-ness" looks like so well that you can create a new cat that doesn't exist.

2. Generative Adversarial Networks (GANs)

- **The Core Idea:** A game of "Cops and Counterfeiters".
- **The Generator (G):** The counterfeiter. It tries to create fake data (images) from random noise (z) to fool the discriminator.
- **The Discriminator (D):** The cop. It tries to classify images as "Real" (from the dataset) or "Fake" (from the generator).
- **The Loop:**
 1. G makes a fake.
 2. D guesses if it's real or fake.
 3. D gets better at spotting fakes.
 4. G gets better at making fakes to beat the improved D .
 - **Equilibrium:** Eventually, G creates perfect fakes, and D can't tell the difference (predicts 0.5 probability).

3. DCGAN (Deep Convolutional GAN)

- Making GANs work with images is hard (unstable training). DCGAN introduced architectural rules to stabilize it:
 - **No Pooling:** Use strided convolutions (discriminator) and fractional-strided convolutions (generator) instead.
 - **Batch Norm:** Use Batch Normalization in both networks.
 - **ReLU:** Use ReLU in Generator (except output which is Tanh) and Leaky ReLU in Discriminator.

- **No Fully Connected Layers:** Use global average pooling or direct connections.

Part 2: The Generated Solved Problem (Step-by-Step)

Since the PDF did not have a specific exercise, I have designed one based on the core mathematical concept of GANs: **The Minimax Loss**.

Exercise 10

We want to calculate the loss for the **Discriminator** and the **Generator** in a single training step.

Given:

- **Real Image (x):** A sample from the real dataset.
- **Noise (z):** A random noise vector.
- **Generator Output ($x_{fake} = G(z)$):** The fake image created by the generator.
- **Discriminator Output on Real ($D(x)$):** Probability that real image is real. Let's say $D(x) = 0.9$.
- **Discriminator Output on Fake ($D(G(z))$):** Probability that fake image is real. Let's say $D(G(z)) = 0.2$.

Task:

1. Calculate the **Discriminator Loss (L_D)**.
2. Calculate the **Generator Loss (L_G)**.

Solution:

Step 1: The Discriminator Loss (L_D) The Discriminator wants to maximize the probability of correctly identifying real images as 1 and fake images as 0. The loss function (binary cross-entropy) is:

$$L_D = -[\log(D(x)) + \log(1 - D(G(z)))]$$

- **Log(Real):** $\log(0.9) \approx -0.105$
- **Log(Fake):** $\log(1 - 0.2) = \log(0.8) \approx -0.223$
- **Sum:** $(-0.105) + (-0.223) = -0.328$
- **Negative:** $L_D = 0.328$.
- **Intuition:** The discriminator is doing a good job (0.328 is low). It correctly assigns high probability to real (0.9) and low to fake (0.2).

Step 2: The Generator Loss (L_G) The Generator wants to *fool* the discriminator. It wants to maximize $D(G(z))$. Standard Loss (Minimizing): $L_G = -\log(D(G(z)))$.

- **Calculation:**

$$L_G = -\log(0.2) \approx -(-1.609) = 1.609$$

- **Intuition:** The loss is high! The generator failed to fool the discriminator (who only gave it a 0.2 score). The generator needs to update its weights significantly to lower this loss.
- **Source:** **Page 9** defines the discriminator minimization objective:
 $-y \log D(x) - (1 - y) \log(1 - D(x))$. **Page 11** defines the generator minimization objective:
 $-\log(D(G(z)))$.

Part 3: Your Turn (Interactive Exercises)

Practice Problem 1: Transposed Convolution (Generator)

DCGAN uses transposed convolutions (often called deconv) to upscale noise into an image. **Setup:**

- **Input Feature Map:** 2×2 .
- **Kernel:** 2×2 , Stride 2.

Task: What is the spatial dimension of the output feature map?

Solving Explanation:

1. **Formula:** Output Size = $(Input - 1) \times Stride + Kernel - 2 \times Padding$. (Assuming Padding=0).
2. **Calculation:** $(2 - 1) \times 2 + 2 = 2 + 2 = 4$.
3. **Result:** The output is 4×4 . It doubled the size.
- **Source:** **Page 19** mentions "Replace any pooling layers with... transposed convolutions (generator)" for spatial upsampling.

Practice Problem 2: GAN Failure Mode

Setup: You are training a GAN. After 10 epochs, you check the generated images. Every single generated image looks exactly the same—a weird grey blob. **Task:** What is this specific failure mode called?

Solving Explanation:

1. **Phenomenon:** The generator finds one specific output that fools the discriminator slightly better than others and keeps producing only that one, losing all diversity.
2. **Name:** This is called **Mode Collapse**.
- **Source:** **Page 17** mentions "preventing the generator from collapsing all samples to a single point, which is a common failure mode observed in GANs."